



PROJECT-BASED LEARNING

PHASE-I EVALUATION

THREADFORGE

Custom Kernel-Level Thread Library

Team ID: T277

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: T277

Semester: 3rd

Domain: *OOP with C++ and DS in C*

Team Members

1. *Manansh Ahuja – 2510011818 (A)*
2. *Aryan Ranjan – 2510011807 (C)*
3. *Aditya Pundir – 2510010372 (K)*
4. *Aman Singh – 2510010366 (B)*

Mentor

Mr. Akshay Rajput

Interactions completed: 5

Problem Statement & Motivation

Problem Statement

Modern operating systems use threads to perform multiple tasks concurrently, but the internal working of thread creation, management, scheduling and synchronization is usually hidden behind standard libraries. This makes it difficult for students to understand thread lifecycle and scheduling. Our project aims to bridge this gap by introducing an educational user-level thread library based on the many-to-one model, using Data Structures in C and OOP in C++.

Motivation

- ▶ Internal working is hidden in standard libraries
- ▶ Difficult to understand user-thread lifecycle and scheduling
- ▶ Helps students observe concurrent task execution

Target Users / Stakeholders

- ▶ Computer Science Students
- ▶ Faculty and instructors teaching Operating Systems
- ▶ Better understanding of user-thread lifecycle and scheduling

Objectives

Project Objectives

1. Study user-level thread libraries and the many-to-one thread model.
2. Design thread control blocks, thread states and thread queues.
3. Implement user-thread creation, execution, yielding, joining, sleeping and termination using C++ classes.
4. Implement a simple user-level scheduler using a ready queue.
5. Implement basic synchronization mechanisms.
6. Track user-thread states, scheduling and execution.

Key Objective: To understand and demonstrate the scheduling and management of multiple user threads using one kernel thread.

Proposed Solution

- ▶ Multiple user-level threads run over a single kernel-level thread.
- ▶ Create, join, yield, sleep and terminate user threads.
- ▶ Maintain TCBs, individual stacks, thread states and IDs.
- ▶ Use queues and scheduling logic to decide which thread runs next.
- ▶ Switch between user threads and provide mutex-based synchronization.
- ▶ C/C++ & Linux Integration

We will manage multiple user-level threads, while Linux provides the underlying kernel level execution.

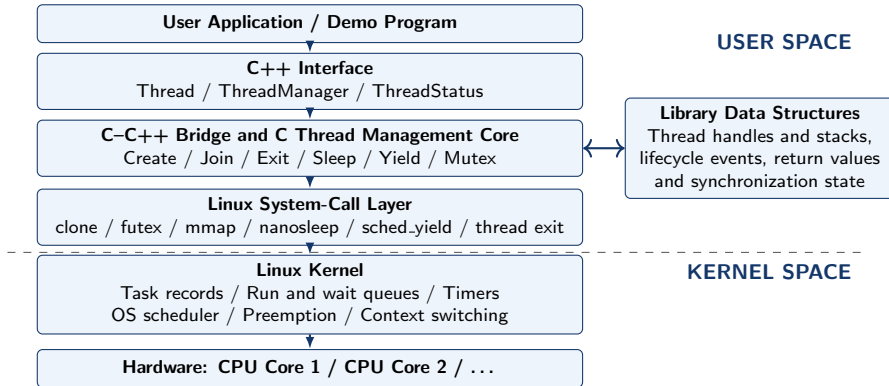
Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Thread records, arrays or linked lists, handle lookup and stack tracking	C thread registry
OOP with C++	Classes, encapsulation and object lifetime management	Thread, ThreadManager and ThreadStatus
Operating Systems	Shared address space, kernel scheduling, blocking and synchronization	System-call layer and mutex implementation

Course Integration

C data structures manage library metadata. C++ classes expose that functionality through a reusable interface.

System Architecture / Workflow



Technology Stack & Methodology

Technology Stack

- ▶ Linux system calls for thread execution
- ▶ C and atomics for our thread library
- ▶ C++ for the object interface
- ▶ GCC / G++ for compilation
- ▶ VS Code and Git / GitHub

Development Methodology

1. Define API, thread records and ownership
2. Build stack setup and clone entry path
3. Implement join, exit, sleep and yield
4. Build mutexes and C++ classes
5. Test races, cleanup and concurrent work

Implementation scope: our own C functions on Linux system calls. Linux provides scheduling and context switching.

Initial Progress & Team Contribution

Progress Achieved

- ▶ *Background study completed*
- ▶ *Requirements identified*
- ▶ *System Architecture prepared*
- ▶ *Modules and milestones planned*

Individual Contribution

Member	Role	Contribution
Manansh Ahuja	Lead	Team Coordination
Aditya Pundir	Research	Background Research
Aman Singh	Design	Architecture Planning
Aryan Ranjan	Documentation	PPT Documentation

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: API and architecture design
2. Phase-II: C lifecycle and mutexes
3. Phase-III: C++ classes and validation

Expected Outcomes

- ▶ Custom C library on Linux system calls
- ▶ Simple C++ thread management
- ▶ Tested mutex and cleanup behavior
- ▶ Lifecycle and multicore demonstrations

References

1. Akshay Rajput, "Thread Library" (course reference).
2. Linux man-pages: clone(2) and futex(2).
3. Linux man-pages: mmap(2), nanosleep(2), sched_yield(2) and exit(2).
4. GNU GCC documentation: atomic operations.

Thank You

Questions & Suggestions